FINAL 5-DAY STAGE-4 SPRINT — DYNAMIC SCRAPING DOMINATION 🚀

Day 0 — Dynamic X-Ray (30–90 min per target)

Goal: Decide how the site actually delivers data so you pick the right tool before coding.

What to do

- Open DevTools → **Network** (filter XHR/Fetch) → **Elements**.
- Classify the site as one of: `HTML / XHR API / GraphQL / Cursor-pagination / Token-auth`.
- Write in notebook: `Site Type | Data Source | Auth Type | Best Tool` (e.g. API → aiohttp; heavy JS + login → Playwright + hybrid).

Why: saves hours. If API exists → use it; browser last-resort.

Failure modes

- Miss hidden API → you waste browser work.
- Assume static when it's GraphQL.

Done when: you can answer "how data is delivered" in ≤10 minutes for a new site.

Day 1 — Playwright fundamentals (codegen → manualize)

Goal: Drive the browser like a human and produce resilient automation.

Tasks

1. Install & test:

```
python -m pip install playwright
playwright install
playwright codegen https://example.com
```

2. Record flows with `codegen`. Convert generated script into resilient selectors:
    - Prefer `get_by_role`, `get_by_text`, or `locator('[data-attr="value"]')`.
    - Avoid brittle XPaths like `/div[3]/span[2]`.

3. Add basic tracing & screenshots on failure.

Why: Codegen teaches actions; manualize makes it robust.

Failure modes

- Using time.sleep → flaky
- Using deep XPaths → breaks on layout change

Done when: you can automate search → click → extract **without** any `time.sleep`.

Day 2 — Waits, scrolls, stop conditions (stability)

Goal: Make infinite scroll and dynamic loads reliable and safe.

Tasks

1. Replace sleeps with explicit waits:
    - `page.wait_for_selector()`
    - `page.wait_for_load_state("networkidle")`
2. Implement robust infinite-scroll loop:

```
prev_count = 0
while True:
    page.mouse.wheel(0, 15000)
    page.wait_for_timeout(1500)
    items = page.query_selector_all(".item")
    if len(items) == prev_count: break
    prev_count = len(items)
```

3. Add failure aids:
    - Save screenshot & HTML on exceptions
    - Enable Playwright tracing for one run

Why: Prevent infinite loops and silent failures.

Failure modes

- Scroll triggers lazy loading but returns same DOM nodes (need to detect new item IDs)
- Network jitter → false “no new items”

Done when: scraper reliably stops when no new items appear and recovers with trace info.

Day 3 — Auth & session engineering (login flows)

Goal: Automate login once and reuse session safely.

Tasks

1. Manual/automated login in headed browser and save state:

```
from playwright.sync_api import sync_playwright
with sync_playwright() as p:
    browser = p.chromium.launch(headless=False)
    ctx = browser.new_context()
    page = ctx.new_page()
    page.goto("https://site/login")
    # manual or scripted login...
    ctx.storage_state(path="auth.json")
    browser.close()
```

2. Reuse `auth.json` in production runs: `browser.new_context(storage_state="auth.json")`.

3. Implement session-expiry detection:

    - If response redirects to `/login` or a login selector appears → treat as expired.
    - On expiry: re-login (manual pause or scripted flow), refresh `auth.json`, resume.

Why: Logging every run is slow and fragile; reusing state = stability.

Failure modes

- 2FA/CAPTCHA: don't auto-bypass; pause for manual solve.
- Short token TTL: implement background refresh or logic to re-login automatically.

Done when: authenticated runs survive simulated token expiry and resume after refresh.

Day 4 — Network interception + hybrid bridge (scale)

Goal: Capture the real JSON endpoints inside the browser, then scale extraction using aiohttp.

Tasks

1. In Playwright, listen to responses and find JSON endpoints:

```
def on_response(response):
    ct = response.headers.get("content-type", "")
    if "application/json" in ct:
        try:
            print(response.url, response.json())
        except: pass
page.on("response", on_response)
```

2. Capture headers, cookies, auth tokens (from storage/state or network).

3. Build a hybrid flow:

    - Use Playwright for login & token capture.
    - Export cookies/headers → inject into `aiohttp.ClientSession` for high-volume requests.

Why: Browser finds tokens; async fetches scale fast and cheaply.

Failure modes

- Missing required header (e.g., X-CSRF) when replaying requests → mirror exact headers.
- Token tied to dynamic fingerprint → cannot replay; fallback to browser fetch.

Done when: you can fetch 10× pages with `aiohttp` using browser-derived tokens/headers.

Day 5 — Systemization & capstone (production minimum)

Goal: Turn scripts into a resumable pipeline with logging, checkpoints and validation.

Project (capstone): Authenticated infinite-scroll directory → validated JSON/CSV output (resume after crash).

Structure

```
/project
  /config
  /logs
  /data/raw
  /data/processed
  /auth
  main.py
```

Add

- Retry logic (tenacity or custom with exponential backoff)
- Checkpoint file that stores last page/item id
- Logging with timestamps (rotate log files)
- Validation step (schema check; record bad rows to `data/processed/errors.json`)

Acceptance (Day 5 Done when):

- Pipeline runs end-to-end
- Simulate crash (kill process) → pipeline resumes from last checkpoint
- Session expiry handled automatically and resumes after refresh/login

---

Quick checklist / commands to run *right now*

- Install Playwright:

```
python -m pip install playwright
playwright install
```

- Test codegen:

```
playwright codegen https://example.com
```

- Save auth state (manual login once): (See Day 3 snippet above)

Acceptance criteria — when you move to Stage 5

Move to Stage 5 when you can, consistently without manual fixes:

- Diagnose site type in ≤ 10 minutes.
- Build a working JS-heavy scraper (login → extract → validate) in ≤ 6 hours.
- Resume failed job from last checkpoint and auto-handle session expiry.
